WEEK 03 TECHNICAL REPORT

YCB-V Object Classification I — Baseline MLP

Intern	Mehrana Rezaei
Group	Group 1 — Basic ML Knowledge
Main Quest	Improving Deep Neural Networks
Side Quest	YCB-Video — Object Classification I

1. Objective

The practical objective of Week 3 was to convert the previously explored YCB-V BOP data into a reproducible object-classification problem and establish a clean neural-network baseline. The pipeline selected five object classes, converted bounding-box annotations into RGB object crops, created scene-level train/validation/test splits to reduce frame leakage, trained a simple multilayer perceptron (MLP), and evaluated the resulting model with training curves, test metrics, a confusion matrix, and representative correct and incorrect predictions.

The baseline was intentionally kept simple: no data augmentation, dropout, batch normalization, weight decay, learning-rate scheduling, convolutional layers, or transfer learning were added. This preserves a clear Week 3 reference point for the controlled improvement experiments required in Week 4.

2. Main Quest Progress and Relevant Concepts

The Main Quest for Week 3 is Improving Deep Neural Networks. The Andrew Ng material provided for this week covers dataset splitting, bias/variance diagnosis, regularization, dropout, input normalization, initialization, and gradient-checking concepts. These topics are directly relevant to interpreting the Week 3 baseline and to designing the controlled experiments planned for Week 4.

2.1 Connection between the Main Quest and this notebook

Three concepts were applied directly in the implementation. First, the train/development/test-set lesson was reflected in the scene-level split: complete YCB-V scenes were assigned to only one split, instead of randomly mixing adjacent video frames across train and validation/test. Second, input normalization was applied after resizing each RGB crop to 64×64, mapping each channel to approximately [-1, 1]. Third, bias/variance thinking was used to interpret the training and validation curves rather than treating a single accuracy value as sufficient.

The regularization lessons were intentionally not implemented in the Week 3 baseline. This is useful experimentally: the current MLP provides a fixed reference against which dropout, other regularization methods, augmentation, optimizer changes, and learning-rate changes can be tested one at a time in Week 4. Vanishing/exploding gradients, weight initialization, and gradient checking are important background concepts, but they were not the main limiting factor in this shallow PyTorch MLP; PyTorch autograd handled gradient computation and the default layer initialization was retained.

3. Dataset Preparation

The notebook reused the lightweight YCB-V BOP'19 subset. It parsed RGB paths together with `scene_gt.json` and `scene_gt_info.json`, producing one tabular record for each usable annotated object instance. Object instances with very low visibility or very small bounding boxes were excluded before class selection.

Preparation item	Observed / configured value
Total usable annotated object instances	4,099
Selected-class object instances	1,702
Number of selected classes	5
Minimum visibility threshold	0.35
Minimum bounding-box side	24 px

Preparation item	Observed / configured value
Object crop padding	8%
Final image size	64 × 64 RGB

3.1 Selected classes

Obj. ID	Class	Usable samples	Scenes	Mean visibility
4	tomato_soup_can	427	6	0.957
3	sugar_box	375	5	0.889
1	master_chef_can	300	4	0.900
6	tuna_fish_can	300	4	0.998
12	bleach_cleanser	300	4	0.876

Classes were ranked using scene coverage and usable sample count. Requiring at least three scenes per class helped ensure that each selected class could appear in train, validation, and test without placing the same scene in multiple splits.

4. Method and Implementation

Stage	Implementation
Dataset loading	Downloaded/reused ycbv_base.zip and ycbv_test_bop19.zip; parsed BOP JSON and RGB images.
Instance table	Built rows containing scene/image IDs, object ID/name, bounding box, visibility and RGB path.
Class selection	Selected the five classes with strongest scene coverage among eligible classes.
Crop creation	Cropped each object from the RGB frame using its annotated bounding box plus 8% padding.
Split strategy	Assigned entire scenes to train/validation/test; asserted pairwise scene disjointness.
PyTorch dataset	Resize to 64×64, ToTensor, normalize each channel with mean=0.5 and std=0.5.
Model	Flattened RGB input → 256-unit ReLU layer → 128-unit ReLU layer → 5 class logits.
Training	Cross-entropy loss, Adam optimizer, learning rate 0.001, batch size 64, 12 epochs.
Model selection	Saved the checkpoint with the lowest validation loss.
Evaluation	Accuracy, per-class precision/recall/F1, confusion matrix, correct/wrong prediction examples.

5. Experimental Setup

5.1 Train / validation / test split

Split	Samples	Scenes	Class distribution
Train	877	8	tomato 202; sugar 225; master chef 150; tuna 150; bleach 150
Validation	375	2	75 samples for each class
Test	450	2	tomato 150; all other classes 75 each

No scene leakage was detected: the train, validation, and test scene-ID sets were pairwise disjoint. This is important because neighboring video frames are highly correlated and a random frame split could produce an unrealistically easy validation result.

5.2 Baseline architecture and training setup

Item	Value
Input	3 × 64 × 64 normalized RGB crop
Flattened input dimension	12,288
Hidden layer 1	Linear(12,288 → 256) + ReLU
Hidden layer 2	Linear(256 → 128) + ReLU
Output layer	Linear(128 → 5)
Trainable parameters	3,179,525
Loss	CrossEntropyLoss
Optimizer	Adam
Learning rate	0.001
Batch size	64
Epochs	12
Device	CPU

6. Results

Metric	Observed result
Best epoch (lowest validation loss)	2
Best validation loss	0.0516
Best validation accuracy	98.13%
Test accuracy	51.33%
Macro precision / recall / F1	0.55 / 0.52 / 0.52
Weighted precision / recall / F1	0.55 / 0.51 / 0.52

6.1 Representative visualizations

The following crops are actual outputs from the executed notebook. They confirm that the annotation-to-crop pipeline works, but they also show substantial variation in viewpoint and context. Some crops include parts of neighboring objects because the crop is derived from a rectangular bounding box with padding.



Figure 2. Representative RGB object crops from the selected five classes.

The selected classes exhibit large pose variation: sugar boxes appear both upright and horizontal; tuna cans are often seen from a top/side viewpoint; bleach cleanser appears with different rotations; and several crops contain nearby objects. This makes the task more challenging for a fully connected MLP because flattening the image does not provide translation or local-pattern invariance.

6.2 Training behavior

Training converged extremely quickly. Train accuracy rose from 91.0% at epoch 1 to 99.9% at epoch 2 and 100% from epoch 3 onward. Validation accuracy peaked at 98.13% at epoch 2. Validation loss was also lowest at epoch 2 (0.0516), then increased and stabilized around 0.067 while training loss was effectively zero.

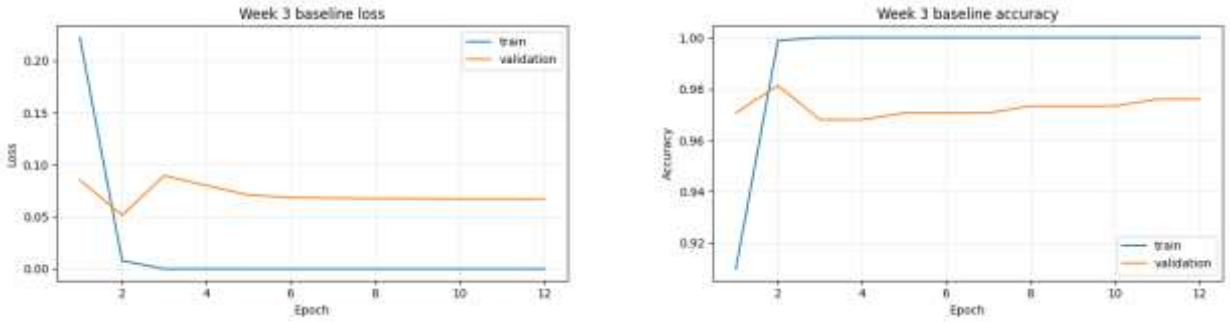


Figure 3. Training/validation loss (left) and accuracy (right) across 12 epochs.

The curve pattern indicates that the MLP has enough capacity to memorize the training subset very early. The rise in validation loss after epoch 2 is an early overfitting signal, even though validation accuracy remains high. The best checkpoint therefore correctly occurs at epoch 2 rather than at the final epoch.

6.3 Test-set classification metrics

Class	Precision	Recall	F1	Support
tomato_soup_can	0.55	0.47	0.51	150
sugar_box	0.67	0.39	0.49	75
master_chef_can	0.90	1.00	0.95	75
tuna_fish_can	0.51	0.60	0.55	75
bleach_cleanser	0.10	0.15	0.12	75
Macro average	0.55	0.52	0.52	450
Weighted average	0.55	0.51	0.52	450

The 51.33% test accuracy is much lower than the 98.13% best validation accuracy. Therefore, the validation split was not a reliable estimate of performance on the two held-out test scenes. The result is consistent with a strong scene/domain effect: the model learned the training scenes very well, and the two validation scenes were apparently easier or more similar to training than the test scenes.

6.4 Confusion matrix

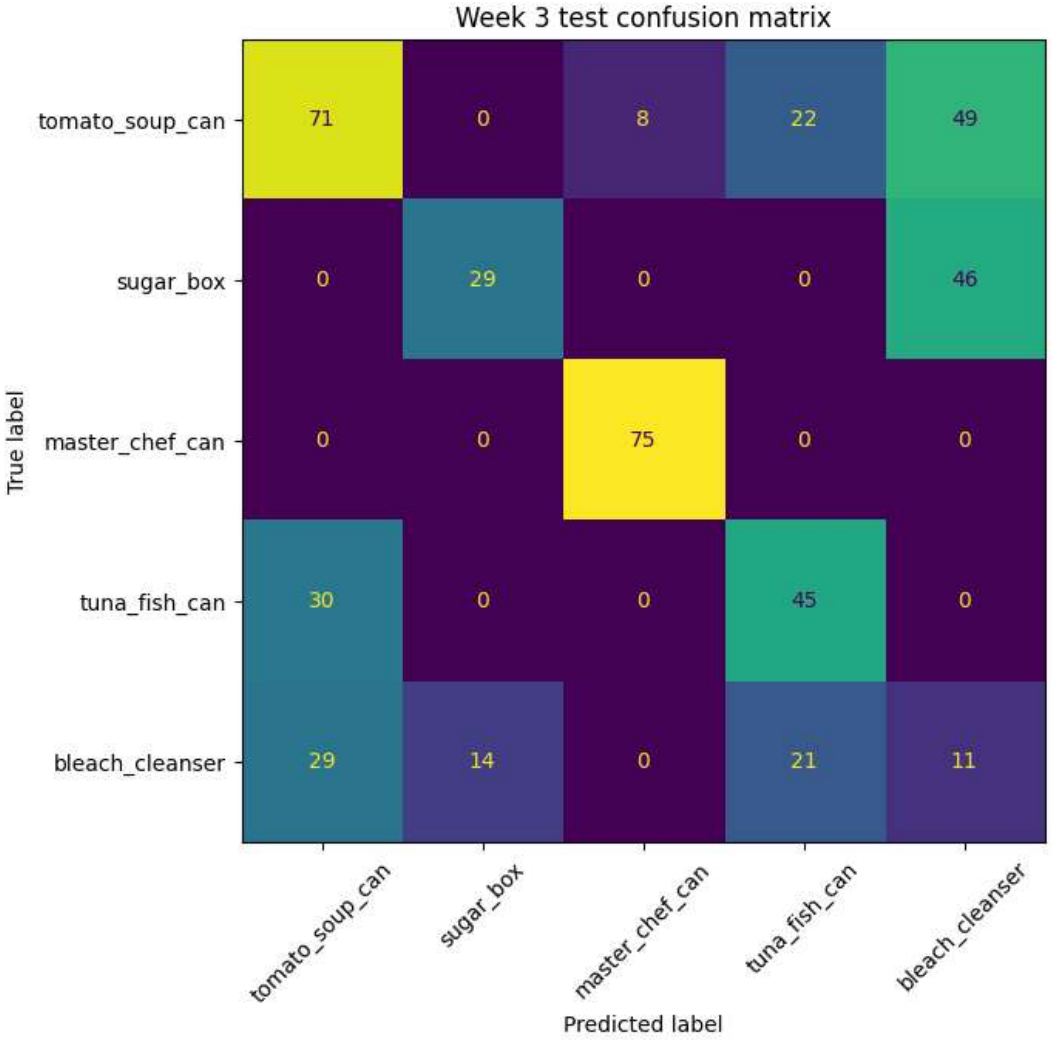


Figure 4. Test-set confusion matrix for the five selected classes.

True class	Correct / support	Main confusions
tomato_soup_can	71 / 150	49 → bleach; 22 → tuna; 8 → master chef
sugar_box	29 / 75	46 → bleach
master_chef_can	75 / 75	No errors in this test split
tuna_fish_can	45 / 75	30 → tomato
bleach_cleanser	11 / 75	29 → tomato; 21 → tuna; 14 → sugar

Master Chef Can was the strongest class, with 100% recall. Bleach Cleanser was the weakest class, with only 15% recall and substantial confusion with tomato soup, tuna, and sugar. Sugar Box was also frequently predicted as Bleach Cleanser. These errors dominate the gap between validation and test performance.

6.5 Representative predictions

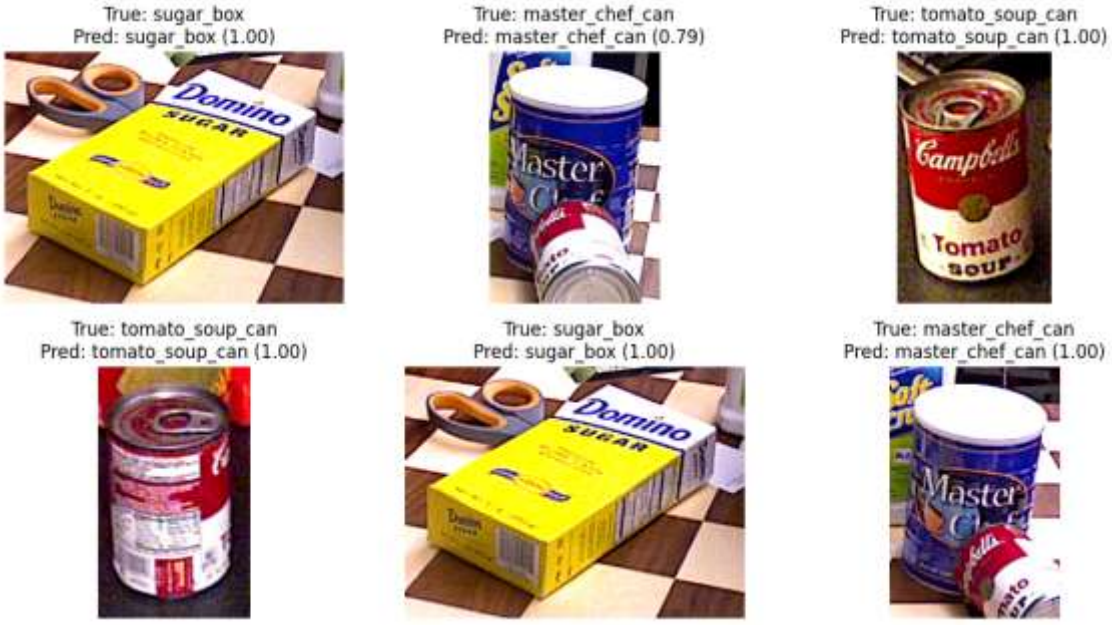


Figure 5. Representative correct test predictions from the executed notebook.

Several correct examples have very high confidence, especially the strongly textured sugar and tomato-soup crops. Master Chef Can is also recognized reliably despite partial occlusion in some examples.



Figure 6. Representative incorrect test predictions from the executed notebook.

The failure examples support the confusion-matrix pattern. Multiple bleach-cleanser crops are predicted as tomato soup, sugar, or tuna with very high confidence; one tuna can is predicted as tomato soup with confidence 1.00. The confidence of these wrong predictions shows that the problem is not merely uncertainty at the decision boundary. The baseline has learned features that are strongly associated with the training/validation scenes but do not transfer reliably to the held-out test scenes.

7. Failure Analysis and Engineering Interpretation

- Early memorization: training accuracy reached 100% by epoch 3 and training loss became essentially zero.
- Validation/test mismatch: best validation accuracy was 98.13%, but test accuracy was only 51.33%.
- Limited scene diversity: the final split used only 2 validation scenes and 2 test scenes, so the result is sensitive to which scenes are assigned to each split.
- Context leakage inside crops: padded rectangular crops can contain neighboring objects and background patterns, allowing the MLP to learn scene/context cues instead of object identity alone.

- Lack of spatial inductive bias: flattening a 64×64 image discards the 2D local structure that CNNs exploit, making viewpoint and location changes harder to generalize across.
- Class-specific weakness: bleach_cleanser had only 15% recall, while master_chef_can reached 100% recall, showing that the error is highly class dependent rather than uniform.

The most important interpretation is that the Week 3 result should not be described as a 98% classifier. The relevant unseen-scene result is the 51.33% test accuracy. The high validation score is still useful because it reveals a weakness in the current experimental split: the validation scenes did not represent the test difficulty. This observation is itself a valuable engineering result and provides a clear motivation for controlled robustness experiments and stronger validation design.

8. Conclusion and Week 4 Plan

Week 3 successfully produced the first end-to-end YCB-V classification baseline. The notebook converted BOP annotations into 1,702 object crops across five classes, enforced a scene-level split with no scene leakage, and trained a reproducible 3.18-million-parameter MLP. The model fit the training data almost perfectly and reached 98.13% validation accuracy at the best checkpoint, but only 51.33% test accuracy. The large validation/test gap shows that generalization across held-out scenes is the primary issue, not the ability to optimize the training set.

For Week 4, the same baseline configuration and split should be kept as the reference experiment while changes are introduced one at a time. The most useful controlled experiments are dropout or weight decay, data augmentation, optimizer comparison, and learning-rate tuning. Because the validation split appears optimistic, Week 4 should also track test-independent robustness evidence carefully and, if allowed by the project rules, consider an additional scene-level cross-validation or alternative fixed split as a secondary diagnostic rather than silently replacing the Week 3 benchmark.